



Assignment No-01

Assignment on Understanding structured programming in C

Course Name: Structured Programming

Course Code: CSE-121

Submitted to

Rishad Amin Pulok

Senior Lecturer

Dept. of Computer Science & Engineering

Metropolitan University, Sylhet

Submitted by

Mahjabin Jahan Anika

ID: 251-115-083

Batch: 62

Section: B

Dept. of Computer Science & Engineering

Metropolitan University, Sylhet

Submission Date: 2025-07-20

Purpose of the assignment: The purpose of this assignment is to show a clear understanding of structured programming and how it is used in the C programming language. It also focuses on writing programs using basic concepts like variables, data types, conditions, and loops in an organized way.

Objectives:

- ① To learn the fundamental syntax and features of C.
- ② To use conditional statements and loops.
- ③ To solve problems using C code.
- ④ To enhance logical thinking and flow control in programming.

Definition of structured programming:

Structured programming is a method of writing programs using clear and organized code with control structures like sequence, selection, and loops. It avoids goto statements and breaks the programs into smaller parts, making it easier to read, test, and maintain.

Importance of structured programming:

- ① Makes programs easier to read and understand.

- ② Helps in faster development and teamwork.
- ③ Improves the overall quality of code.

Overview of the C language:

C is a powerful and flexible general-purpose programming language developed by Dennis Ritchie in the 1970s.

Key features:

It can be defined by the following ways:

- ① Mother language.
- ② System programming language.
- ③ Procedure-oriented programming language.
- ④ Structured programming language.
- ⑤ Mid-level programming language.

Tokens in C programming:

Tokens are the smallest individual units in a C program.

Types of Tokens:

- 7 Keywords: int, float, return, if, while
- 7 Identifier: sum, main (variable or function names)
- 7 Constants: 10, 3.14, 'A'

7 Operators: +, -, *, /, ==

7 Special symbols: (), {}, [], ;

7 Strings: "Hello", "C language"

Data Types (DT):

- ▶ Primitive data type/built in data type
- ▶ Derived data type
- ▶ User defined data type

⇒ Primitive data type:

Integer	Float	Double	Character
int	float	double	char
2-4 bytes	4 bytes	8 bytes	1 byte
10, 20, ...	10.234	10.23456	'A', '1'
%d	%f	%lf	%c

⇒ Derived data type:

Derived data type	Description	Example syntax
Array	Collection of elements of the same type stored in continuous memory locations	int numbers [5]; char name [20]
Pointer	Variables that stores the memory address of another variable	int *ptr; ptr = #
Function	A block of reusable code that performs a specific task and can return a value	int add(int a, int b); void display();
String	A sequence of characters (technically a character array ending with '\0')	char name [] = "Maktabin";

⇒ User defined data types:

data type	Description	Example syntax
Struct	Groups variable of different data types into a single unit	C\nstruct Person {\n int age;\n char name [20];\n}
Union	Shares memory among all members	C\nunion Data {\n int i;\n float f;\n}

data type	Description	Example Syntax
enum	Defines a set of named integer constants	C\n enum Day { Sun, MON, TUE};
typedef	Creates a new name for an existing data type.	C\n typedef int Number; \n Number x = 10;

Variable: It is a name given to a memory location where data is stored. It can be changed during program execution.

Example:

```
int age = 20;
float pi = 3.1416;
char grade = 'A';
```

Operator: It is a symbol that performs operations on variable and values.

⇒ Types of operators:

- On basis of number of operands
- On basis of the way of operations

⇒ On the basis of number of operands :

operator	Works on	Example
Unary operator	1 operand	&, ++, --, &marks
Binary operator	2 operands	+, -, /, *, &&, 5+2
Ternary operator / conditional operator	3 operands	? : Expression 1 ? Expression 2 : Expression 3

⇒ On the basis of the way of operations :

1 Arithmetic operator : + - * / %

1 Assignment operator : = += -=

1 Relational operator : == != > < >= <=

1 Logical operator : &&

1 Bitwise operator : ~ | & ^ << >>

1 Special operator : sizeof & * → .

Use of different operators with examples:

⇒ Arithmetic Operators: Used for mathematical operations.

Operator	Meaning	Example	Example ($a=10, b=5$)
+	Addition	$a+b$	15
-	Subtraction	$a-b$	5
*	Multiplication	$a*b$	50
/	Division	a/b	2
%	Modulus	$a \% b$	0

Example:

```
int a=10, b=5;  
printf("Multiplication = %.d\n", a*b);
```

⇒ Relational Operators: Used to compare two values.

Operator	Example	Output
==	$a == b$	0
!=	$a != b$	1
>	$a > b$	1
<	$a < b$	0
>=	$a >= b$	1
<=	$a <= b$	0

Example:

```
int a=5;
int b=3;
int result=a>b;
printf("%d", result); // output: 1
```

⇒ Logical Operators: Used to combine multiple conditions.

A	B	!A	!B	A & B	A B
0	0	1	1	0	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	1	1

Example:

```
int main() {
    int a=5, b=10;
    // Logical AND
    if(a<10 && b>5)
        printf("AND condition is true \n");
    // Logical OR
    if(a==5 || b==0)
        printf("OR condition is true \n");
}
```

```
// Logical NOT
```

```
if (! (a == b))
```

```
printf("NOT condition is true\n");
```

```
return 0;
```

```
}
```

⇒ Assignment Operators: Used to assign values to variables.

Operator	Example	Equivalent to
=	$a = 10$	Assign 10 to a
+=	$a += 5$	$a = a + 5$
-=	$a -= 5$	$a = a - 5$
*=	$a *= 2$	$a = a * 2$
/=	$a /= 2$	$a = a / 2$
%=	$a \% = 2$	$a = a \% 2$

⇒ Bitwise Operator: These operate on bits.

Operator	Meaning
&	AND
	OR
^	XOR
<<	Left shift
>>	Right shift

Example:

```
int a=5, b=3;  
int result = a & b; // bitwise AND  
printf("Result = %.d\n", result);  
// output: 1
```

Program 1: Check Even or Odd using if-else.

Source Code:

```
#include <stdio.h>  
int main () {  
    int num;  
    scanf ("%d", &num);  
  
    if (num % 2 == 0)  
        printf ("%d is Even\n", num);  
    else  
        printf ("%d is Odd\n", num);  
  
    return 0;  
}
```

Sample Input : 6

Output : 6 is Even

Program #28 Find Greatest of Three Numbers using
Nested if-else

Source Code:

```
#include <stdio.h>
int main () {
    int a, b, c;
    printf("Enter three numbers:");
    scanf("%d%d%d", &a, &b, &c);
```

```
    if (a > b)
    { if (a > c)
      printf("%d\n", a);
      else
        printf("%d\n", c); }
```

```
    else
    { if (b > c)
      printf("%d\n", b);
      else
        printf("%d\n", c); }
```

```
    return 0;
```

```
}
```

Sample Input: 5 9 3

Output: Enter three numbers: 5 9 3

9

Problem 3: Simple calculator using switch case.

Source Code:

```
#include <stdio.h>
```

```
int main () {
```

```
    int a, b; char op;
```

```
    scanf("%d %d", &a, &b);
```

```
    printf("Enter operator (+, -, *, /): ");
```

```
    scanf("%c", &op);
```

```
    switch (op) {
```

```
        case '+':
```

```
            printf("Result: %d\n", a+b); break;
```

```
        case '-':
```

```
            printf("Result: %d\n", a-b); break;
```

```
        case '*':
```

```
            printf("Result: %d\n", a*b); break;
```

```
        case '/':
```

```
            printf("Result: %d\n", a/b); break;
```

```
default:
    printf("Invalid operator.\n"); }

return 0;
}
```

Sample Input: 10 5

Output: ^{10 5}
Enter operator (+, -, *, /): +

Result: 15

Problem - 9: Print multiplication table using for loop.

Source Code:

```
#include <stdio.h>
int main () {
    int num;
    printf("Enter a number:");
    scanf("%d", &num);

    for (int i=1; i<=10; i++)
        printf("%d X %d = %d\n", num, i, num*i);
    return 0;
}
```

Input : 3

Output :

Enter a number : 3

$$3 \times 1 = 3$$

$$3 \times 2 = 6$$

$$3 \times 3 = 9$$

$$3 \times 4 = 12$$

$$3 \times 5 = 15$$

$$3 \times 6 = 18$$

$$3 \times 7 = 21$$

$$3 \times 8 = 24$$

$$3 \times 9 = 27$$

$$3 \times 10 = 30$$

Problem-5: Print digits in reverse.

Source Code: #include <stdio.h>

```
int main () {
```

```
int num;
```

```
printf("Enter a number:");
```

```
scanf("%d", &num);
```

```
printf("Digits in reverse:");
```

```
while (num > 0)
```

```

    { printf("%d", num % 10);
      num /= 10; }
    printf("\n");
    return 0;
}

```

Sample Input: 1234

Output: Enter a number : 1234
 Digits in reverse: 4321

Problem-6: Sum until user enters zero using do-while loop.

Source Code:

```

#include <stdio.h>

int main () {
    int num, sum = 0;
    do { printf("Enter a number (0 to stop): ");
        scanf("%d", &num);
        sum += num; }
    while (num != 0);
    printf("Total sum: %d\n", sum);
}

```

```
return 0;  
}
```

Input: 5 10 -3 0

Output: Enter a number (0 to stop): 5 10 -3 0
Total sum: 12

Summary of learning outcomes:

- ① Gained practical experience using control structures in C.
- ② Understood how to guide program flow using conditions and loops.
- ③ Developed logic-building skills for real-world scenarios.
- ④ Learned how to read user input and process it accordingly.

References:

- Let US C by Yashavant Kametkar.
- Tutorialspoint:

<https://www.tutorialspoint.com/cpprogramming/>

- This assignment is written based on class lecture notes, along with information from online programming tutorials.